Building an Afaan Oromoo Language Model from Scratch: Data, Tokenization, Pretraining and Adaptation on a Laptop

Habtamu Feyera
Independent researcher

Abstract

We build a complete language-model pipeline for Afaan Oromoo, a widely spoken but under-resourced Cushitic language, on a single laptop (Apple M1 Max). Following the from-scratch recipe of Raschka (2025), we adapt each stage to Oromo: an 8.07M-word cleaned corpus that adds public Telegram news channels to AfriBERTa and Wikipedia; a byte-level BPE tokenizer that keeps the hudhaa (') inside words and halves sequence length relative to GPT-2 (4.98 vs. 2.34 characters per token); and a 23M-parameter GPT pretrained in 46 minutes to 1.52 bits per character. Fine-tuned for news-topic classification on MasakhaNEWS, the pretrained model reaches 87.2% accuracy and 0.779 macro-F1, against 0.558 for the same architecture trained from random weights and 0.749 for TF-IDF with logistic regression. Instruction tuning on 3,758 examples built from real Oromo text teaches the 23M model the answer format (100% valid topic labels) but not to use its input; the same data with LoRA on an Oromo-adapted 0.8B model reaches 91.1% topic accuracy and beats a lead baseline on headlines. Comparing open models in bits per character, our 23M model outperforms a general 752M multilingual model (1.51 vs. 2.88), while the same 752M model after African-language continued pretraining is best (1.45; 1.02 on posts published after its release). LoRA continued pretraining of Qwen3-0.6B on 2.5M Oromo tokens cuts its bits per character by 43%. The results point to Oromo-specific data and tokenization, more than model size, as the main levers for small-scale Oromo language modeling. Code: https://github.com/HabtamuFeyera/afaan-oromoo-llm.

1 Introduction

Afaan Oromoo is one of the most widely spoken languages in Africa, yet it has little digital text and few language models. General-purpose tokenizers and models are built mostly from English and other high-resource languages. They split Oromo words into many fragments and often fail to produce Oromo at all: prompted with *Afaan Oromoo*, a general 752M-parameter model continued in Afrikaans in our tests.

This paper asks how far a single researcher with a laptop can get by building every stage for Oromo from the ground up. We follow the open, from-scratch GPT recipe of Raschka (2025) and change what Oromo needs changed: the data, the tokenizer, the model size, the evaluation and the instruction data. We then compare the result with open multilingual models. We address four questions:

1. **Tokenization:** how much does an Oromo-specific tokenizer save over GPT-2's English-centric one?
2. **Pretraining at small scale:** what can a 23M-parameter model learn from about 13M tokens of Oromo?
3. **Transfer:** does that pretraining help downstream tasks, and does instruction tuning work at this scale?
4. **Data versus size:** how does a small Oromo-only model compare with larger open models, before and after Oromo adaptation?

Our contributions are:

- A reproducible cleaning pipeline for Oromo text that unifies the hudhaa, removes news-site boilerplate and checks every evaluation set for overlap with the training data.
- A byte-level BPE tokenizer that keeps the hudhaa inside words, with a vocabulary-size study.
- A 23M-parameter Oromo GPT evaluated on language modeling, topic classification and

instruction following, with baselines and multiple seeds.

- A tokenizer-independent comparison (bits per character) with general and Africa-adapted open models, including our own LoRA continued pretraining.

- An instruction dataset built from real Oromo text rather than translation, and an analysis of why instruction tuning fails at 23M parameters.

2 Related Work

Language models for African languages. AfriBERTa (Ogueji et al., 2021) showed that small encoder models pretrained only on African languages can match much larger multilingual models; its corpus is our main pretraining source. MasakhaNEWS (Adelani et al., 2023) provides news-topic classification for 16 African languages, including Oromo, which we use for downstream evaluation. AfriqueLLM (Yu et al., 2026) continues pretraining open decoder models on African languages, Oromo included; we use its 0.8B model as a reference point.

Tokenization. Byte pair encoding (Sennrich et al., 2016) and its byte-level form in GPT-2 (Radford et al., 2019) learn subwords from frequency, so languages rare in the training data are split into short fragments. An existing 12k byte-level BPE tokenizer for Oromo trained on AfriBERTa (NaolBM) is our tokenizer baseline.

Small models and limited data. Our pretraining recipe follows Raschka (2025). Muennighoff et al. (2023) show that repeating data for up to about four epochs costs little compared with fresh data, which motivates our four passes over a fixed corpus.

Contamination. We follow the 13-gram overlap test of Brown et al. (2020) to check evaluation sets against pretraining data.

Instruction tuning and adaptation. We build instruction data from existing labeled and structured text, as FLAN did (Wei et al., 2022; Longpre et al., 2023), and use an Alpaca-style prompt (Taori et al., 2023). For adapting larger models we use LoRA (Hu et al., 2022).

Source	Docs	Words
AfriBERTa, Oromo subset	sentences	6.54M
Oromo Wikipedia (Nov 2023)	1,772	0.36M
Telegram channels (2019–2026)	10,845	1.15M
Total		8.07M

Table 1: Training text by source. AfriBERTa is Apache-2.0 and Wikipedia CC BY-SA 4.0. Telegram posts (OBN, TIKVAH Afaan Oromoo, Jawar Mohammed, Oromia Times) remain under their channels' copyright and are used locally only.

3 Data

The final pretraining corpus has 8.07M words of cleaned Oromo text from three sources (Table 1), plus held-out validation sets that no training text overlaps.

Cleaning. Every line is normalized and filtered by the same rules. *Hudhaa:* the raw AfriBERTa text writes the glottal-stop apostrophe two ways almost equally often (189k ASCII ' and 173k U+2019), so a model would see two spellings of words like *baay'ee*; all apostrophe variants are mapped to ASCII '. *Boilerplate:* links, Wikipedia citation marks, BBC photo-caption labels (*Madda suuraa*, *Ibsa waa'ee suuraa*) and site menus are removed, keeping the surrounding sentence. *Filters:* lines are dropped if they have fewer than 3 words, contain Cyrillic or Arabic script, are more than 10% Ethiopic, are mostly non-letters, or are mostly English function words. *Deduplication:* exact duplicates are removed, ignoring case and punctuation; validation text is processed first, so a repeated line is always removed from training. A sample of every dropped rule is written out for review; a first pass dropped real sentences that began with a caption label, ended in an ellipsis or quoted a short English phrase, and fixing these rules recovered about 2% of the text.

Telegram channels. Public channel histories were exported with Telegram Desktop. Hashtags, handles, emojis and links are removed. Any line that appears in three or more posts of a channel is treated as a signature and removed: 58,612 lines from OBN alone, mostly social-media link blocks, sign-offs and reporter bylines. Telegram text is appended last, so earlier outputs are unchanged.

Contamination checks. Any Telegram post that shares a single 13-word sequence with the validation set or the MasakhaNEWS dev and test articles

Tokenizer	Vocab	C/T↑	T/W↓	1-tok↑	Rare↓
GPT-2	50,257	2.34	3.29	7.5	91.2
Oromo-BBPE	12,000	4.59	1.68	66.7	7.9
Ours 8k	8,000	4.47	1.73	64.4	1.5
Ours 16k	16,000	**4.98**	**1.55**	**73.8**	2.8
Ours 24k	24,000	5.24	1.47	78.5	4.5
Ours 32k	32,000	5.41	1.42	81.5	29.5

Table 2: Tokenizers on the validation text. C/T: characters per token; T/W: tokens per word; 1-tok: % of word occurrences that are a single token; Rare: % of the vocabulary seen fewer than 50 times in training.

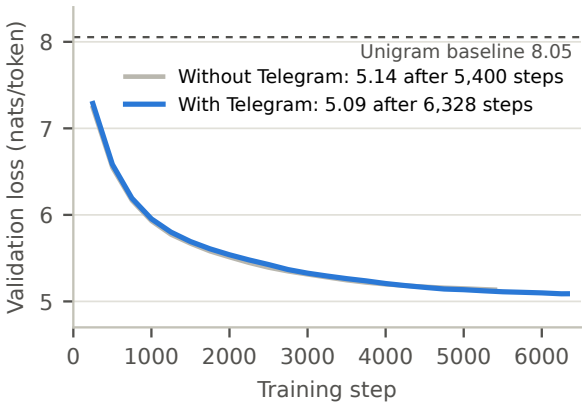


Figure 1: Validation loss falls from 7.3 to 5.1 and is still falling when training stops. Loss on 40 validation batches every 250 steps.

is dropped (15 posts). In the other direction, only 1 of 325 MasakhaNEWS test articles has at least half of its 13-grams in the pretraining text; on average 5.5% do. Headlines are usually shorter than 13 words, so we also matched them as whole phrases (ignoring case and punctuation): 81 of the 325 test headlines appear verbatim in the pretraining text, all of them in AfriBERTa. Topic labels are not part of the text, so this affects only the headline task, for which we also report scores on the 244 unseen headlines (§7, §8).

Splits. Validation consists of the AfriBERTa evaluation split and 5% of Wikipedia articles (0.51M words). A separate Telegram validation set holds 5% of posts (65k words).

4 Tokenization

An Oromo byte-level BPE tokenizer with a 16k vocabulary halves sequence length relative to GPT-2 (Table 2).

Design. We train byte-level BPE, as in GPT-2, so no input is out of vocabulary. GPT-2's pre-tokenizer always splits at an apostrophe, which in Oromo cuts words such as *deebi'uun* and *ta'a* in two. Our pre-tokenizer keeps the hudhaa inside a word (`?\p{L}+(?:'\p{L}+)*`, followed by GPT-2's number, punctuation and whitespace patterns) and still separates quote marks at word edges. The sentence *Afaan Oromoo afaan ummata Oromoo kan Gaanfa Afrikaa keessa jiraatuuti.* takes 28 tokens with GPT-2 and 12 with ours.

Vocabulary size. We choose 16k. Moving to 24k shortens sequences by only 5% but adds 50% more embedding parameters. At 32k, 29.5% of the vocabulary is seen fewer than 50 times, too rarely for a small model to learn. Our 16k tokenizer also compresses Oromo 8% better than the existing 12k Oromo tokenizer. With it, the training corpus is 12.97M tokens, against 27.3M with GPT-2.

5 Pretraining

Model. We use the GPT architecture of Raschka (2025, ch. 4) unchanged: pre-LayerNorm transformer blocks with GELU feed-forward layers, learned absolute position embeddings and no weight tying, with 6 layers, 6 heads, width 384, a 256-token context (about 1,270 characters) and 23.0M parameters. Attention uses PyTorch's fused scaled-dot-product kernel, which we test to be numerically identical to the book's mask-and-softmax code. We train with AdamW (weight decay 0.1, peak learning rate 6×10^{-4}, 2% linear warmup, cosine decay to 10%, gradient clipping at 1.0) on batches of 32×256 tokens for four passes over the corpus, at about 18k tokens/s on an Apple M1 Max GPU.

Runs. The first run trains on the corpus without Telegram (11.07M tokens, 5,400 steps, 42 minutes); the second adds the Telegram data (12.97M tokens, 6,328 steps, 46 minutes). Both keep the checkpoint with the lowest validation loss. Validation loss improves at every checkpoint (Figure 1); the final gap between training and validation loss is 0.23 and 0.31, so neither run overfits and more data or longer training should still help.

Results. Table 3 reports perplexity and bits per character. Adding 17% more text, all newer news, cuts bits per character on held-out Telegram posts by 15% and still improves the original validation set by 1%. Two caveats apply: the second run also took 17% more optimizer steps, and Telegram val-

	Validation		Telegram	
Model	PPL	BPC	PPL	BPC
Unigram	3,147	2.41	–	–
GPT, no Telegram	171	1.536	217	1.516
GPT, + Telegram	**163**	**1.522**	**96**	**1.286**

Table 3: Language modeling on held-out text. Perplexity is per token of our tokenizer; bits per character (BPC) is tokenizer-independent.

idation posts come from the same channels as the training posts, so recurring templated posts make that set easier than new text.

Samples. Generations at temperature 0.7 are fluent-looking news Oromo with correct hudhaa forms and quoting style, but their meaning drifts: *Mootummaan naannoo Oromiyaas ta'e kan naannoo Benishaangul Gumuz, naannoo Amaaraa, magaalaa Naqamtee fi magaalota naannoo Sumaalee jiraatan keessatti* . . . At temperature 0 the model loops (*Afaan Oromoo, Afaan Oromoo,* . . .). Of the words generated at temperature 0.7, 93% occur in the training text.

6 Topic Classification

Task. MasakhaNEWS Oromo assigns BBC Afaan Oromoo articles to five topics, with 1,128 training, 162 dev and 325 test articles. Topics are unbalanced (technology has only 50 training articles), so we report macro-F1 alongside accuracy. The model reads the headline and the start of the article, cut to the 256-token context.

Method. Following Raschka (2025, ch. 6), the 16k-word output layer is replaced by a 5-class head, and the class is read at the last real token. We compare a majority-class baseline; TF-IDF word uni- and bigrams with logistic regression (C tuned on dev); and the GPT trained from random initialization, from pretraining with only the last block, final norm and head trained (the book's recipe, 1.8M parameters), and from pretraining with all layers trained (16.9M parameters). Each GPT setup tries learning rates $\{5\times10^{-5}, 2\times10^{-4}, 5\times10^{-4}\}$ for up to 10 epochs, keeps the best epoch on dev macro-F1, and reruns the best rate with 3 seeds. These experiments use the pretrained model without Telegram.

Results. Pretraining is what makes the model competitive (Figure 2): macro-F1 rises from 0.558 to 0.779 and accuracy from 73.1% to 87.2%, edging out TF-IDF (0.749, 86.8%). The gain is largest on rare topics (Table 4): technology F1 rises from 0.11 to 0.58, above TF-IDF's 0.42, because the model has already seen their vocabulary in unlabeled text. TF-IDF still wins on sports, where team and player names decide the topic. Training a tenth of the parameters (the book's recipe) recovers about half of the gain over random initialization. Removing the one test article found in the pretraining data changes no score at three decimals.

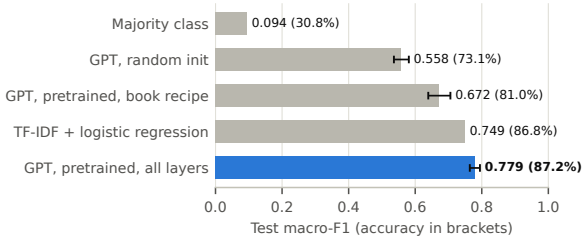


Figure 2: Pretraining raises test macro-F1 from 0.56 to 0.78, edging out TF-IDF (0.75). GPT systems: mean of 3 seeds, whiskers ±1 SD.

Topic (train)	TF-IDF	GPT rand.	GPT pretr.
Sports (280)	**0.98**	0.90	0.93
Health (338)	0.88	0.80	**0.91**
Politics (350)	0.87	0.76	**0.90**
Entertainment (110)	**0.60**	0.23	0.58
Technology (50)	0.42	0.11	**0.58**

Table 4: F1 per topic (GPT systems: all layers trained, mean of 3 seeds).

7 Instruction Tuning

Data. No Oromo instruction dataset exists, and translating an English one would bring in translation errors. We therefore build tasks from real Oromo text, as FLAN did; every answer is human-written Oromo, and only the instruction templates (two per task) were written for this work. *Topic* asks for the topic word of a MasakhaNEWS article (*Oduun armaan gadii waa'ee maalii? Tokko filadhu: siyaasa, fayyaa, ispoortii, bashannanaa, teeknooloojii.*); *headline* asks for its real headline (*Oduu armaan gadiitiif mata duree gabaabaa barreessi.*); *describe* asks for the first sentence of an entity's Wikipedia article (*Waa'ee {X} ibsa gabaabaa kenni.*). This gives 3,758 training, 384 validation and 731 test examples; news items follow the MasakhaNEWS splits, and test *describe* items come from Wikipedia articles held out of pretraining.

Task	Metric	Before	After
Topic	valid label	0%	**100%**
Topic	acc. / macro-F1	0% / 0.00	50.5% / 0.29
Headline	ROUGE-L	0.005	0.025
Describe	ROUGE-L	0.024	0.079

Table 5: Instruction following of the 23M model on the test set (greedy decoding). References: the topic classifier of §6 reaches 87.2% / 0.78 and the majority class 30.8%; copying an article's first 10 words as its headline scores ROUGE-L 0.212.

Method. We use the recipe of Raschka (2025, ch. 7) with an Alpaca-style prompt whose section names are in Oromo (`### Qajeelfama:`, `### Galtee:`, `### Deebii:`). Prompt tokens are masked out of the loss, and articles are cut so the example fits 256 tokens. Of two learning rates and five epochs, the best validation loss came at 5×10^{-5} after 2 epochs (6.45 to 4.05); at 2×10^{-4} the model memorized the training set.

Results. Instruction tuning teaches the format but not the use of the input (Table 5). Topic answers are almost always *fayyaa* or *ispoortii* and never *teeknooloojii*. 240 of 325 generated headlines mention Covid-19, against 53 of the real ones: the model falls back on the most common kind of training headline. Having seen a headline in pretraining barely helps: ROUGE-L is 0.045 on the 81 test headlines that appear in the pretraining text and 0.018 on the other 244. Descriptions of unseen entities are invented, often describing a district (*Aanaan* . . .) even for a country. The same weights classify topics at 87% with a classification head, so the information is present; turning it into instruction following appears to need more capacity, which is why Raschka (2025) uses the 355M GPT-2 for this step.

8 Comparison with Open Models

Models and measurement. We compare Qwen3.5-0.8B-Base (Qwen Team, 2026), a general multilingual base model (752M parameters); AfriqueQwen3.5-0.8B (Yu et al., 2026), the same model after continued pretraining on African languages including Oromo; Qwen3-0.6B-Base (Yang et al., 2025) before and after our own LoRA continued pretraining; and our GPT with Telegram data. All models score the same text in bits per character. Each gets about 1,300 characters of context (256 tokens for ours, 512 for the Qwen tokenizers, which average 2.5 characters per

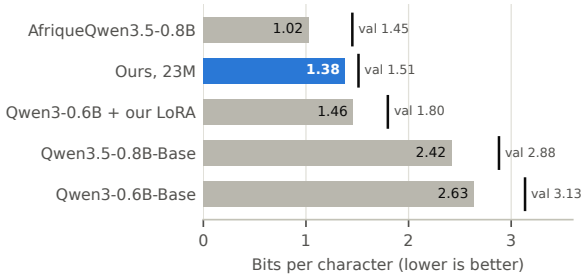


Figure 3: Oromo data beats size. Bars: bits per character on 28 Telegram posts from 2026 that no model saw; ticks: validation set.

Oromo token), and a sliding window scores every character with at least half a window of context. Test sets are the first 150k characters of the validation set and of the held-out Telegram posts, and 28 held-out posts (25k characters) published after AfriqueLLM's January 2026 release, which no model here can have seen. AfriqueLLM's training data may include text like our validation set, which comes from AfriBERTa and Wikipedia.

Out-of-the-box Oromo. The general Qwen3.5 model is far behind our 23M model on every set (2.88 vs. 1.51 bits per character on validation; Figure 3); prompted with *Afaan Oromoo*, it continues in Afrikaans. After African-language continued pretraining, the identical architecture produces fluent Oromo and is the best model on every set; its margin over ours is largest on the unseen 2026 posts (1.02 vs. 1.38).

Our own continued pretraining. We continued pretraining Qwen3-0.6B-Base on 2.5M tokens of our corpus with LoRA: rank-64 adapters on every attention and MLP projection (40M trainable parameters), 55 minutes at about 800 tokens/s. Validation bits per character fell 43%, from 3.14 to 1.80, and similarly on Telegram (2.63 to 1.48). The adapted model still trails our 23M model, which saw the full corpus four times, and AfriqueQwen, which was adapted with far more data.

Instruction tuning at 0.8B. To test whether the failure of §7 is one of scale, we instruction-tune AfriqueQwen3.5-0.8B on the same data with the same prompt and prompt masking. LoRA adapters of rank 16 on all attention, linear-attention and MLP projections (10.2M trainable parameters) train for one epoch at learning rate 2×10^{-4}, with examples cut to 384 tokens (about 950 characters). Training took about an hour; validation loss on answer tokens is 1.30.

Task	Metric	23M	0.8B
Topic	accuracy	50.5%	**91.1%**
Topic	macro-F1	0.29	**0.87**
Headline	ROUGE-L	0.025	**0.254**
Describe	ROUGE-L	0.079	**0.186**

Table 6: Instruction following on the full test set: our 23M model (§7) against AfriqueQwen3.5-0.8B with LoRA instruction tuning on the same data. Copying an article's first 10 words as its headline scores ROUGE-L 0.212.

The larger, Oromo-adapted model follows instructions and uses its input (Table 6). Topic accuracy rises from 50.5% to 91.1% and macro-F1 from 0.29 to 0.87, with F1 of at least 0.73 on every topic (0.80 on technology). Its headlines beat the copy-the-first-10-words baseline (0.254 vs. 0.212). AfriBERTa is public and may be part of AfriqueLLM's pretraining data, so we also score the 244 test headlines that do not appear in it: the margin holds (0.255 vs. 0.218). Only 58 of 325 mention Covid-19, against 53 of the real ones. Descriptions of unseen entities improve but remain the weakest task; 3 of 81 degenerate into repetition under greedy decoding. Before instruction tuning, on the first 150 test articles, the same model never produced a valid topic label, although its headline continuations already scored ROUGE-L 0.135. Topic accuracy exceeds our classifier of §6 (87.2%), but AfriqueLLM's pretraining corpus may contain BBC Afaan Oromoo articles like the MasakhaNEWS test set, which we cannot check, and this result is from a single run.

Compute note. Training Qwen3.5 on a Mac ran at only 92–290 tokens/s: its linear-attention layers have fast kernels only for NVIDIA GPUs, so PyTorch falls back to a slow reference implementation. We therefore used Qwen3 for our own continued pretraining and kept Qwen3.5 to inference and one epoch of instruction tuning.

9 Discussion

For small-scale Oromo modeling, the evidence points to data and tokenization before model size. English-built tokenizers spend about twice as many tokens on Oromo (2.3–2.6 characters per token, against 5.0 for ours), which halves the text a fixed context window can hold. A 23M model trained only on Oromo beats a general 752M model by almost 2× in bits per character; once the larger model receives Oromo-rich continued pretraining, it becomes the strongest. Pretraining transfers to topic classification, most strongly for topics with few labeled examples. Instruction following needs more capacity than 23M parameters: the small model learns the format but not to read its input, while an Oromo-adapted 0.8B model tuned on the same 3,758 examples follows the instructions and uses its input.

10 Conclusion

A single laptop is enough to build a working Oromo language-modeling pipeline, and the largest gains come from Oromo-specific data and tokenization. A hudhaa-aware 16k tokenizer halves sequence length; a 23M-parameter GPT trained in under an hour outperforms a general 752M model on Oromo and, fine-tuned, matches a strong TF-IDF baseline on topic classification. Instruction following does not emerge at this scale, but the same instruction data turns an Oromo-adapted 0.8B model into a working Oromo instruction follower on a laptop. Next steps are native-speaker evaluation, more data beyond news, multi-seed instruction tuning of larger Oromo-adapted models, broader benchmarks that include Oromo such as Belebele and SIB-200, and a public release of the tokenizer and of model weights trained on redistributable data.

Limitations

The test sets are small (325 news articles, 81 descriptions, 28 posts in the unseen 2026 set), so differences of a few points can be noise; we report seed spreads where we can. Generated text and the instruction templates have not been rated by native speakers, and ROUGE-L against one reference is a weak measure of headline quality. The data is mostly news, and Telegram validation posts come from the same channels as the training posts. All training ran on one laptop: pretraining used four passes over the corpus, our continued pretraining saw only a fifth of it, and the faster Qwen3.5 kernels exist only for NVIDIA GPUs. The run with Telegram data also took 17% more optimizer steps, and AfriqueQwen and our adapted Qwen3 start from different base models. The 0.8B instruction-tuning result is a single run of one epoch, and AfriqueLLM's pretraining corpus may overlap with the MasakhaNEWS test articles, which we cannot check. A quarter of the test headlines (81 of 325) appear in our pretrain-

ing text; future corpora should remove evaluation headlines before training.

Ethics Statement

All training text comes from public sources. We release code, the trained tokenizer and the cleaning rules (Apache-2.0); AfriBERTa (Apache-2.0), Oromo Wikipedia (CC BY-SA 4.0) and MasakhaNEWS (AFL-3.0) are downloaded by the scripts. Telegram posts remain under their channels' copyright and are not released, nor are weights trained on them; the released importer lets others rebuild the corpus from their own exports. The data is mostly news and includes reporting on conflict, politics and public health; one channel is a politician's and carries opinion rather than reporting. Posts can name public figures and private individuals, and we did not test the models for memorization of individual posts. The models are research artifacts for low-resource language modeling and are not suitable for producing news, health information or any content presented as fact.

Acknowledgments

This work was carried out with substantial assistance from an AI assistant (Claude, by Anthropic), which wrote much of the code and a first draft of the text under the author's direction. The author directed the project, collected the Telegram data, reviewed the results and takes full responsibility for the content.

References

David Ifeoluwa Adelani et al. 2023. MasakhaNEWS: News topic classification for African languages. In *Proceedings of the 13th International Joint Conference on Natural Language Processing and the 3rd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics (IJCNLP-AACL)*.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, et al. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*.

Shayne Longpre, Le Hou, Tu Vu, Albert Webson, Hyung Won Chung, Yi Tay, Denny Zhou, Quoc V. Le, Barret Zoph, Jason Wei, and Adam Roberts. 2023. The Flan collection: Designing data and methods for effective instruction tuning. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *PMLR*, pages 22631–22648.

Niklas Muennighoff, Alexander M. Rush, Boaz Barak, Teven Le Scao, Nouamane Tazi, Aleksandra Piktus, Sampo Pyysalo, Thomas Wolf, and Colin Raffel. 2023. Scaling data-constrained language models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

NaolBM. Oromo-BBPE: a byte-level BPE tokenizer for Afaan Oromo. Hugging Face model repository. Accessed November 2026.

Kelechi Ogueji, Yuxin Zhu, and Jimmy Lin. 2021. Small data? no problem! exploring the viability of pretrained multilingual language models for low-resourced languages. In *Proceedings of the 1st Workshop on Multilingual Representation Learning*, pages 116–126.

Qwen Team. 2026. Qwen3.5: Towards native multimodal agents. Blog post.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language models are unsupervised multitask learners. Technical report, OpenAI.

Sebastian Raschka. 2025. *Build a Large Language Model (From Scratch)*. Manning, Shelter Island, NY.

Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1715–1725.

Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. Stanford Alpaca: An instruction-following LLaMA model. GitHub repository.

Jason Wei, Maarten Bosma, Vincent Y. Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai, and Quoc V. Le. 2022. Finetuned language models are zero-shot learners. In *International Conference on Learning Representations (ICLR)*.

An Yang et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

Hao Yu, Tianyi Xu, Michael A. Hedderich, Wassim Hamidouche, Syed Waqas Zamir, and David Ifeoluwa Adelani. 2026. AfriqueLLM: How data mixing and model architecture impact continued pre-training for African languages. *arXiv preprint arXiv:2601.06395*.

A Reproducibility

Every result comes from numbered scripts in the public repository, run on one Apple M1 Max with 64 GB of memory: data preparation (`01`–`03`, `14`; about 1 minute), tokenizers and encoding (`04`–`06`; 2 minutes), pretraining (`07`; 42–46 minutes), its evaluation (`08`, `15`), topic classification (`09`, `10`; 30 minutes), instruction tuning (`11`–`13`; 15 minutes), open-model comparison (`16`; 6 minutes), LoRA continued pretraining (`17`; 55 minutes), LoRA instruction tuning and its full-test evaluation (`18`, `20`; about 2.5 hours) and the figures (`19`). Software: PyTorch 2.12 (MPS), Hugging Face `tokenizers` 0.22, `transformers` 5.9 and `peft` 0.21. Pretraining uses seed 123 and classification seeds 1–3; cleaning, splits and sampling are deterministic. The repository's 16 tests cover the tokenizer, data loaders, the model's attention against the book's reference code, and the classification and instruction-tuning data handling. Telegram text must be exported by each user; without it the pipeline skips that source and reproduces the results without Telegram.